

Upcoming changes to the MFEM Memory Manager

MFEM Community Workshop 2026 – Atlanta, GA

September 23, 2026

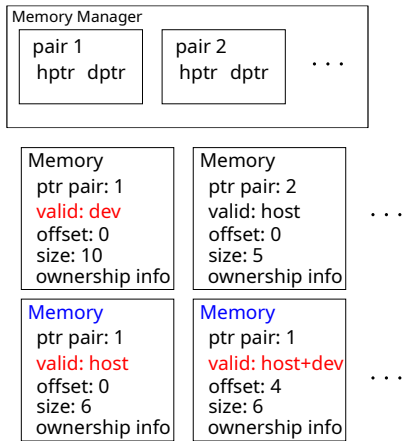
Andrew Ho



Overview

- Overview of the MFEM memory manager
- Key design changes
 - Automatic syncing of aliases
 - CheckedWrap
 - New memory spaces
 - Temporary workspace memory spaces
 - Umpire managed memory pools

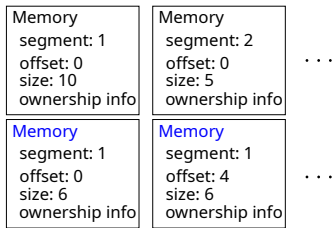
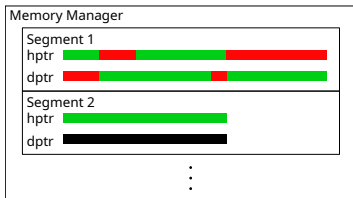
The MFEM memory manager manages host/device pointer pairs



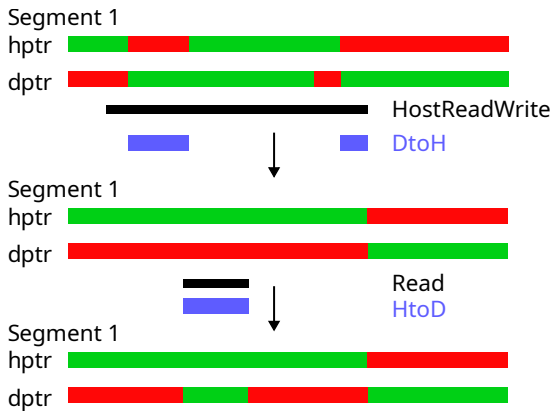
Current: the user is responsible for updating validity flags with aliases

```
1  mfem::Array<int> offsets(3);
2  offsets[0] = 0; offsets[1] = V1size; offsets[2] = V1size + V2size;
3  mfem::BlockVector State(offsets);
4  mfem::Vector &V1 = State.GetBlock(0), &V2 = State.GetBlock(1);
5  // ... code
6  State.SyncFromBlocks(); // <-- currently: Don't forget to do this!
7  auto s = State.HostReadWrite();
8  for (int i = 0; i < V1size+V2size; ++i) { s[i] += i; }
9  // ... code
10 v1.SyncMemory(State); // <-- currently: Don't forget to do this!
11 auto v1 = V1.ReadWrite();
12 mfem::forall(V1size, [=]MFEM_HOST_DEVICE(int i) { v1[i] += 2*i;});
13 // ... code
```

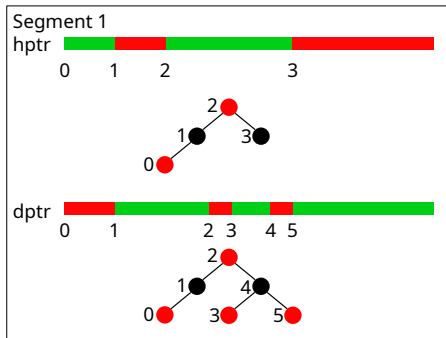
Automatic sync requires tracking fragmented validity



Read/write operations color portions of a segment as valid/invalid



Segment validity transitions tracked using a red-black tree



- Transition markers automatically merged/removed if possible
- Performance scales with $O(\log(\text{num transitions}))$

Behavior change: Any copy in SyncToAlias is deferred

```
1  mfem::Vector base(10);
2  base = 3.1;
3  mfem::Vector a1(base, 0, 5);
4  mfem::Vector a2(base, 5, 5);
5  auto d1 = a1.ReadWrite(), d2 = a2.ReadWrite();
6  mfem::forall(5, [=] MFEM_HOST_DEVICE(int i) {d1[i] += i; d2[i] *= i;});
7  // current: two copies
8  a1.SyncAliasMemory(base);
9  a2.SyncAliasMemory(base);
10 // current: 3.1 4.1 5.1 6.2 7.2 0 3.1 6.23 9.3 12.4
11 // new: error, copy from device to host is deferred
12 for(int i = 0; i < 10; ++i) { std::cout << base[0] << " "; }
13 std::cout << std::endl;
14 // new: copy is coalesced
15 base.HostRead();
16 // now both will print out the same expected result
17 for(int i = 0; i < 10; ++i) { std::cout << base[0] << " "; }
18 std::cout << std::endl;
```

Unchecked wrapping aliased pointers double registers the pointer

```
1  void RunUnchecked(mfem::real_t *ptr, int size) {
2      // ptr must NOT alias a registered pointer of any kind
3      mfem::Vector v(ptr, size);
4      auto d_ptr = v.Write();
5      mfem::forall(size, [=] MFEM_HOST_DEVICE(int i) { d_ptr[i] = 4 - i; });
6  }
7
8  void MyRunner()
9  {
10     mfem::Vector v1(4);
11     v1 = 0.0;
12     auto d_ptr = v1.ReadWrite();
13     mfem::forall(2, [=] MFEM_HOST_DEVICE(int i) { d_ptr[i] += i; });
14     v1.Print(); // [0, 1, 0, 0]
15     RunUnchecked(v1.HostReadWrite() + 2, 2);
16     v1.Print(); // [0, 1, 0, 0], expected [0, 1, 4, 3]
17     d_ptr = v1.ReadWrite();
18     mfem::forall(2, [=] MFEM_HOST_DEVICE(int i) { d_ptr[i+2] += 1; });
19     v1.Print(); // [0, 1, 1, 1], expected [0, 1, 5, 4]
20 }
```

CheckWrap can wrap aliased pointers

```
1  void RunChecked(mfem::real_t *ptr, int size) {
2      // ptr can alias a registered pointer of any kind
3      mfem::Vector v(ptr, size, true);
4      auto d_ptr = v.Write();
5      mfem::forall(size, [=] MFEM_HOST_DEVICE(int i) { d_ptr[i] = 4 - i; });
6  }
7
8  void MyRunner()
9  {
10     mfem::Vector v1(4);
11     v1 = 0.0;
12     auto d_ptr = v1.ReadWrite();
13     mfem::forall(2, [=] MFEM_HOST_DEVICE(int i) { d_ptr[i] += i; });
14     v1.Print(); // [0, 1, 0, 0]
15     RunUnchecked(v1.HostReadWrite() + 2, 2);
16     v1.Print(); // [0, 1, 4, 3]
17     d_ptr = v1.ReadWrite();
18     mfem::forall(2, [=] MFEM_HOST_DEVICE(int i) { d_ptr[i+2] += 1; });
19     v1.Print(); // [0, 1, 5, 4]
20 }
```

New: Temporary memory spaces

```
1  auto temp_j = mfem::Array<int>::NewTemporary(10,  
    mfem::Device::GetDeviceMemoryType());  
2  auto temp_v = mfem::Vector::NewTemporary(10,  
    mfem::Device::GetDeviceMemoryType());  
3  mfem::GridFunction temp_f0(fespace,  
    mfem::GetTemporaryMemorySpace(mfem::Device::GetDeviceMemoryType()));  
4  mfem::GridFunction temp_f1(fespace, mfem::MemoryType::TEMP_MANAGED);
```

For Umpire users: DEVICE_UMPIRE_2, TEMP_DEVICE_UMPIRE, and TEMP_DEVICE_UMPIRE_2 all map to the same umpire memory pool.

New: Umpire managed memory pools

```
1  // Optional if you want to use your own umpire pool
2  auto& rm = umpire::ResourceManager::getInstance();
3  auto mypool = rm.makeAllocator<umpire::strategy::QuickPool>("MyManagedUmpire",
    rm.getAllocator("UM"));
4  auto mytmppool =
    rm.makeAllocator<umpire::strategy::QuickPool>("MyManagedUmpireTemp",
    rm.getAllocator("UM"));
5  mfem::MemoryManager::SetUmpireManagedAllocatorName("MyManagedUmpire");
6  mfem::MemoryManager::SetUmpireTempManagedAllocatorName("MyManagedUmpireTemp");
7  // Configure both host and device memory to use managed umpire
8  mfem::Device::SetMemoryTypes(mfem::MemoryType::MANAGED_UMPIRE,
    mfem::MemoryType::MANAGED_UMPIRE);
9
10 // Create the device after setting memory types
11 mfem::Device device("raja-gpu");
```

Using managed memory allows MFEM to avoid double allocating separate host and device pointers.

Measuring the performance cost

- Creating a Vector (various memory spaces)
- Wrapping an external pointer (checked and unchecked)
- Application case-study: MARBL 2D triple point

Code being benchmarked

(Unchecked) wrap

```
1  mfem::real_t data[3];
2  mfem::Vector small(data, 3);
3  small(0) = 0; small(1) = 0; small(2) = 0;
```

host

```
1  mfem::Vector small(3);
2  small(0) = 0; small(1) = 0; small(2) = 0;
```

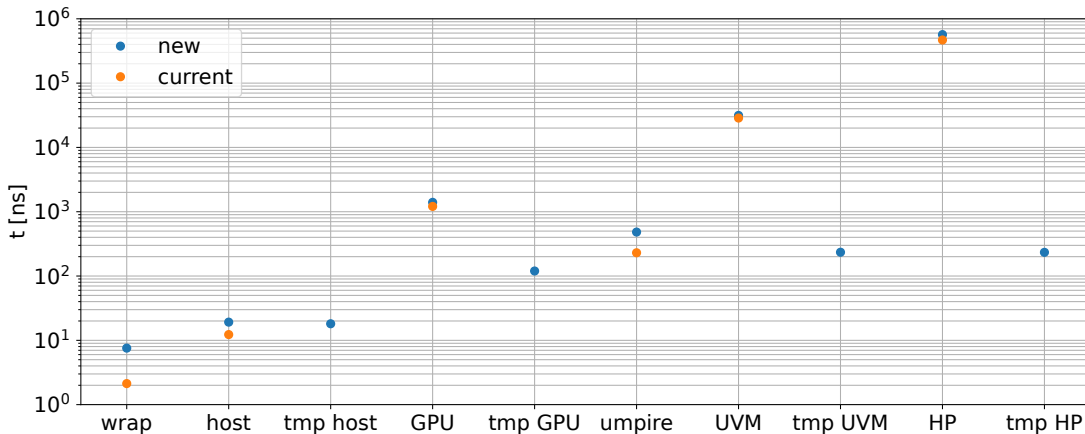
tmp host

```
1  mfem::Vector small(3, mfem::MemoryType::TEMP_HOST);
2  small(0) = 0; small(1) = 0; small(2) = 0;
```

device allocators

```
1  mfem::Vector small(0x10000, memory_type);
2  small.Write();
```

Minimal overhead introduced with new MemoryManager

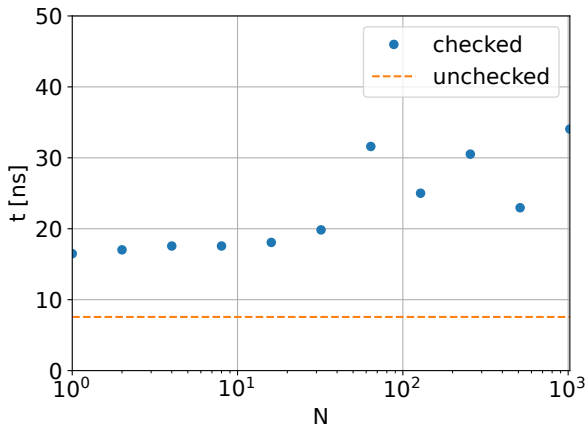


One exception: wrapping an external host pointer

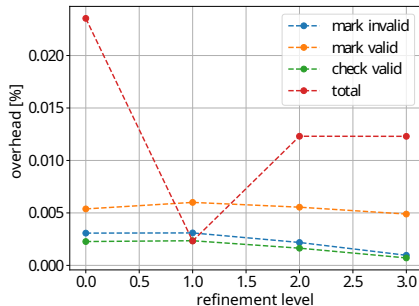
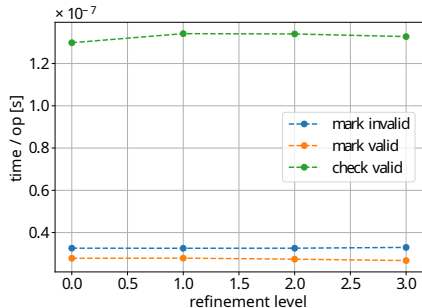
CheckedWrap is $O(\log(N))$ in number of registered entries

```
1  // setup
2  std::vector<mfem::Vector> registered;
3  for (int i = 0; i < N; ++i) {
4      // ensure memory is registered
5      registered.emplace_back(3, mfem::MemoryType::DEVICE);
6  }
7  auto ptr = registered[0].HostWrite();
8  // benchmark
9  mfem::Vector small(ptr, 3, true);
10 small(0) = 0; small(1) = 0; small(2) = 0;
```

CheckedWrap has 100%+ overhead over unchecked



Average overhead in MARBL GPU 2D triple point



- Read requires one mark valid
- ReadWrite/Write requires one mark valid and one mark invalid
- Copy requires 1-2 check valid

Future work

- Multi-thread safety in the new memory manager (with MFEM_THREAD_SAFE). Currently thread safety should mostly match the existing memory manager:
 - Registered aliases are not thread safe
 - Can call read/write on a funneled thread, then pass the pointer to a multithreaded context
 - Temporary memory spaces alloc/dealloc must be funneled
- Update MFEM to internally use temporary workspaces.

<https://github.com/mfem/mfem/pull/5019>

